

Efficient Interpolation-based Pathline Tracing with B-spline Curves in Particle Dataset

Haoyu Li *
The Ohio State University

Tianyu Xiong †
The Ohio State University

Han-Wei Shen ‡
The Ohio State University

ABSTRACT

Particle tracing through numerical integration is a well-known approach to generating pathlines for visualization. However, for particle simulations, the computation of pathlines is expensive, since the interpolation method is complicated due to the lack of connectivity information. Previous studies utilize the k -d tree to reduce the time for neighborhood search. However, the efficiency is still limited by the number of tracing time steps. Therefore, we propose a novel interpolation-based particle tracing method that first represents particle data as B-spline curves and interpolates B-spline control points to reduce the number of interpolation time steps. We demonstrate our approach achieves good tracing accuracy with much less computation time.

Index Terms: Human-centered computing—Visualization—Visualization application domains—Scientific visualization

1 INTRODUCTION

Pathline tracing is a popular technique for flow visualization and analysis, for its ability to depict scientific features in different domains such as aerodynamics and cosmology. Eulerian and Lagrangian flow simulations generate flow fields in two different formats: mesh-based and particle-based. For the meshed-based flow fields, velocities are stored at the vertices of the mesh, while the particle-based method stores the velocity information at particle positions without explicit connectivity defined between particles. For certain particle simulations such as smoothed particle hydrodynamics (SPH), even though the trajectories of the particles already exist, to have a clear view of the underlying features, sometimes it is necessary to have more pathlines around the locations of interest. The widely used numerical integration approaches, such as Euler or Runge-Kutta methods for computing pathlines, require an extensive amount of velocity interpolation both spatially and temporally to obtain reasonable accuracy. The computation time of numerical integration largely depends on the number of integration steps and the complexity of the interpolation method. For large-scale datasets, the I/O overhead can also be very high. Therefore, for particle-based flow fields, using numerical integration to generate pathlines is very expensive.

However, the positions of a particle, or the particle displacement, over time often show particular patterns governed by the underlying physical conditions. Making use of the particle displacement, previous study [3] proposed an interpolation-based tracing approach for particle datasets, where they avoid performing velocity interpolations at every time step needed by numerical integration. Instead, they directly use particle displacement between time steps to generate particle traces. The performance of their method is determined by three factors: neighborhood search time, the number of interpolated pathlines, and the number of interpolation steps. Neighborhood

search time is optimized using a modified k -d tree data structure in their work and pathline advection from different starting points can be parallelized. However, the tracing time for each pathline cannot be easily reduced. Considering that current flow simulations can output data at higher spatial and temporal resolutions, a large number of tracing steps introduces a computational challenge.

To solve the aforementioned challenge, we propose a new approach for interpolation-based pathline tracing. B-spline [6] curves are used first to fit the traces of existing particle data. We optimize the accuracy by using an adaptive knot placement method [16] in B-spline approximation, where more control points are placed at the positions of higher complexity in the curve. Interpolation is performed only between the control points of the parametric curves instead of the original particles, which reduces the number of advection iterations from the number of particle integration steps to the number of control points used. With our method, we demonstrate a significant reduction in the computation cost for particle tracing while achieving similar tracing quality.

2 RELATED WORKS

Most of the approaches to calculating pathlines rely on particle tracing by numerical integration [12, 14]. Since regular grid flow field particle tracing is a well-studied area with mature techniques, while studies on particle-based flow field pathline generation are relatively scarce, we focus on the related works to pathline generation for particle-based flow fields in this section. One of the most relevant studies to our work is interpolation-based pathline tracing [3], where next time step particle location is calculated based on the neighborhood particle position displacement. Their method avoids multiple neighborhood searches needed by numerical integration at each time step. However, their approach faces the problem that the total tracing time is dominated by the number of time steps of the existing pathlines. Particle-based flow fields can also be interpreted as flow maps stored at scattered point locations. Many recent research that use machine learning techniques to generate [7, 10] or super-resolve flow maps [9, 15] are also related to this study. However, machine-learning-based methods usually face the problem of long training time and unbounded tracing errors.

Another related field to our study is using parameter curves to represent pathlines or streamlines in flow visualization. An exploratory research [1] examine different kind of parameter curves and their fitting quality. Hong et al. [8] make use of Bézier curves for flow visualization under a compression-based pathline or streamline reuse framework. Liu and Wang [11] proposed a streamline compression method using B-spline curves that preserves topological relations and has bounded error. Chen et al [4] model pathlines with composite Bézier curves with uncertainty and reduce error using forward and backward trace. In all of these studies, parameter curves show small and controllable fitting errors in representing pathlines and streamlines and fast fitting time.

3 BACKGROUND

In this section, we review the interpolation-based pathline tracing method introduced by Chandler et al. [3] for particle-based flow fields. Each pathline in the dataset can be described as a function: $f_i: \tau \mapsto \rho$, where $\tau \in \mathbb{N}$ is the time step and $\rho \in \mathbb{R}^3$ is the particle

*e-mail: li.8460@osu.edu

†e-mail: xiong.336@osu.edu

‡e-mail: shen.94@osu.edu

position at that time step. When a new particle is inserted at time τ , to compute its trajectory in other time steps, an interpolation-based pathline tracing method described in the following steps is used:

1. Load all particles of time step τ .
2. Find neighbor particles around the inserted particle.
3. Load all particles of time step $\tau + 1$.
4. Calculate the interpolation weights based on the spatial positions of the neighboring particles and the inserted particle at time τ .
5. Reconstruct the position of inserted pathline at time $\tau + 1$ based on neighboring particles' displacement.
6. Update the neighbor particles at time step $\tau + 1$.
7. Repeat from step 3 until the desired time step is reached.

The reconstruction step in 5 can be expressed by the following equation:

$$f_{\text{insert}}(\tau + 1) = \sum_{f_i(\tau) \in N(f_{\text{insert}}(\tau))} w_i(f_i(\tau + 1) - f_i(\tau)) + f_{\text{insert}}(\tau), \quad (1)$$

where we use $N(f_{\text{insert}}(\tau))$ to denote the set of neighboring particles of the particle at position $\rho = f_{\text{insert}}(\tau)$ and w_i to denote the interpolation weight. It is worth noting that the interpolation-based pathline tracing approach is agnostic to the interpolation method and w_i is calculated using the interpolation method of choice, for example, SPH kernels in their study. A modified k -d tree data structure is constructed at every time step to accelerate neighborhood search in their proposed method.

The major limitation of this method is that the total computation time is determined by the number of integration (time) steps needed to calculate the pathline. Our method aims to solve this limitation by applying interpolation-based pathline tracing on B-spline representations of the existing particle traces. The interpolation iteration needed is effectively reduced to the number of control points used to represent the particle traces.

4 METHOD

Our B-spline curve interpolation-based pathline tracing approach can be described at a high level in two steps. First, we process the particle-based flow field and fit a B-spline curve for each existing particle trace. Second, we perform interpolation-based pathline tracing on the control points and knots for a given new particle position and time. We explain these two steps in detail next.

4.1 B-spline Approximation for Particle Traces

A B-spline curve of order k is a piecewise polynomial function of degree $k - 1$ defined by:

$$C(u) = \sum_{i=0}^{n-1} B_{i,k}(u) P_i, \quad u \in [t_0, t_{n+k-1}], \quad (2)$$

where P_i denotes one of the n control points and $C(u)$ evaluates the B-spline curve at parameter location u . Knot vector $T = \{t_0, t_1, t_2, \dots, t_{n+k-1}\}$ defines the parameter u range that is influenced by the control points P . The B-spline basis function $B_{i,k}(u)$ is defined recursively with respect to the knot vector and the curve order k as:

$$B_{i,1}(u) = \begin{cases} 1, & \text{if } t_i \leq u < t_{i+1} \\ 0, & \text{otherwise} \end{cases}, \quad (3)$$

$$B_{i,k}(u) = \frac{u - t_i}{t_{i+k-1} - t_i} B_{i,k-1}(u) + \frac{t_{i+k} - u}{t_{i+k} - t_{i+1}} B_{i+1,k-1}(u).$$

More in-depth content about B-spline curves can be found in this book by Farin [5].

Given a pathline represented by m particles whose positions are $\rho_0, \rho_1, \dots, \rho_{m-1}$, the approximation of a B-spline curve involves three steps: First, we need to parameterize the data points into a monotonically increasing list $u_0, u_1, \dots, u_{m-1}$, which determines the distribution of data points along the B-spline curve. Second, we determine the knot vector along the curve, which decides the control points' distribution in the parameter space. And last, we optimize the control point positions with respect to the parameters and the knot vector as a least-square solution:

$$\arg \min_P \sum_{i=0}^{m-1} \|\rho_i - C(u_i)\|_2 \quad (4)$$

4.1.1 Parameterization

There are two ways to parameterize this trajectory. The first is to use the time steps as parameters: τ as the parameter for ρ . And the second is to use the chord length in 4D space-time to determine a parameter for each particle. We choose to use time steps to parameterize the trajectory for its simplicity. When representing the curve in 4D space, we need to determine how to normalize the spatial dimensions and the temporal dimension. The choice of different normalizations may have an impact on the fitting accuracy. Moreover, parameterizing the pathlines by the 4D chord length also requires us to use 4D control points to describe the B-spline curve, which can be less efficient when applying the interpolation algorithm. Therefore, we normalize the time index to be in the range $[0, 1]$, and each B-spline curve is a function from time to 3D spatial positions.

4.1.2 Knot Placement

Considering a B-spline curve of order k , control point P_i is used when calculating the spline segment between t_i and t_{i+k} . Thus, knot vector $T = \{t_0, t_1, t_2, \dots, t_{n+k-1}\}$ determines the control point density along the pathline in the parameter space. We duplicate the first k and the last k knots to ensure the curve passes through the first and the last control points. Knot placement is a well-known problem in B-spline approximation to optimize the spline quality. We adopted a recent fast automatic knot placement method proposed by Yeh et al. [16] to determine an optimal knot vector for spline fitting.

The automatic knot placement method is inspired by the idea that an order- k B-spline curve has a piecewise constant $(k - 1)$ th derivative [16], and derivative discontinuities mark the knot locations. A feature function is derived from the k th derivative of the data points and used the cumulative feature function to guide the knot placement (more knots when the cumulative feature function changes fast). Their method is empirically found to yield better fitting accuracy than directly using $(k - 1)$ th derivative of the data points.

The only hyper-parameter to choose in automatic knot placement is the number of knots used. Choosing the number of knots also determines the number of control points used for B-spline approximation, since the number of knots is $n + k$ for a B-spline with n control points and order k . This choice is a balance between pathline tracing efficiency and accuracy. We discuss in detail the evaluation of the choice in Sect. 5.1. After a knot vector has been calculated for the given data points, we find the control points of the approximated B-spline curve using the least square method following Equation 4.

4.2 Interpolation-Based B-spline Tracing

Based on the method introduced by Chandler et al. [3], we propose a pathline tracing approach based on B-spline control point interpolation. All particle data are processed first and a B-spline curve C is generated for each particle trajectory. The B-spline approximation only needs to be done once. The control points $P_0, P_1, \dots, P_{n-1}$ and the knot vector $T : t_0, t_1, \dots, t_{n+k-1}$ are saved as the representation for the data instead of the original particle trajectories. Since the automatic knot placement algorithm cannot guarantee that the knots are placed at the same time steps for different pathlines, we need

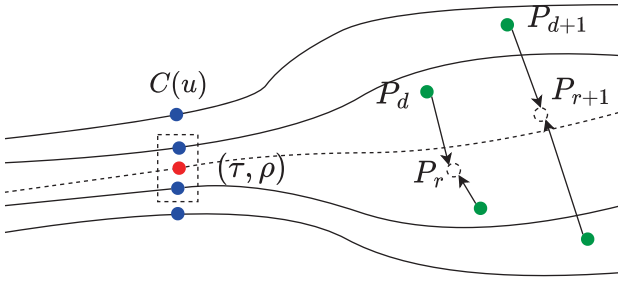


Figure 1: An example for forward tracing of control points. Given the pathline tracing starting point (red dot), we first identify the neighbor curves by evaluating $C(u)$ (blue). We find the nearest knots and control points (green) to $C(u)$ in future time steps. And then neighbor particles (blue dots in the square) and the nearest knots and control points P_d are used to reconstruct knots and control points P_r for the new pathline. We then similarly interpolate knots and control points to get the next iteration knots and control points P_{r+1} for the new pathline.

to synchronize the control points before interpolation. However, for higher efficiency, this synchronization is only performed at the starting point of the traced pathline. For later control points, we simply choose the nearest neighbors among the next control point from each pathline, even though these control points may correspond to the knots of different time steps. The errors introduced by this choice are small for two reasons. First, nearby pathlines will likely share similar shapes so that the knot intervals of nearby pathlines are similar. Second, we interpolate both the knots and the control points, which is similar to interpolating a point in 4D space-time, which eases the problem of unsynchronized time steps.

Next, we describe our algorithm in detail. An illustration of the interpolation scheme can be found in Fig. 1. Given a tracing starting point (τ, ρ) , we present the modified interpolation-based pathline tracing as follows:

1. Normalize τ to get the parameter u and evaluate all existing B-spline at u .
2. For each existing B-spline, find the minimum index $d \in [\lfloor k/2 \rfloor, n+k-1-\lfloor k/2 \rfloor]$ so that $t_d \geq u$. For special cases, where $u = 0$ and 1 , we define $d = \lfloor k/2 \rfloor$ and $n+k-1-\lfloor k/2 \rfloor$.
3. Find neighboring B-splines C and calculate the interpolation weights based on $\|\rho - C(u)\|_2$.
4. Reconstruct the knot t_r and the control point $P_{r-\lfloor k/2 \rfloor}$ of Q based on neighbor t_d and $P_{d-\lfloor k/2 \rfloor}$, where r denotes the knot and control point indices for the interpolated pathlines.
5. Load knots t_{d+1} and control points $P_{d-\lfloor k/2 \rfloor+1}$.
6. Find the neighbor control points and knots. Calculate interpolation weights for iteration r .
7. Reconstruct the knot t_{r+1} and the control point $P_{r-\lfloor k/2 \rfloor+1}$ for the inserted pathline based on the interpolation weights.
8. Update the neighbor knots and control points for iteration $r+1$.
9. Repeat from step 5 until $t_r > 1$.

The steps above describe how we perform forward tracing. Backward tracing, which is necessary to represent the full new B-spline, can be described similarly. The reconstructions in step 4 and step 7 are defined similarly to Equation 1:

$$\begin{aligned}
 t_r &= \sum_i w_i \cdot t_{i,d_i}, \\
 P_{r-\lfloor k/2 \rfloor} &= \sum_i w_i (P_{i,d_i-\lfloor k/2 \rfloor} - C_i(u)) + \rho, \\
 t_{r+1} &= \sum_i w_i (t_{i,d_i+1} - t_{i,d_i}) + t_r, \\
 P_{r-\lfloor k/2 \rfloor+1} &= \sum_i w_i (P_{i,d_i-\lfloor k/2 \rfloor+1} - P_{i,d_i-\lfloor k/2 \rfloor}) + P_{r-\lfloor k/2 \rfloor},
 \end{aligned} \tag{5}$$

where we use i to denote the indices of all neighbors of the inserted pathline. In the first two equations, neighbors are calculated based on the Euclidean distance between the positions of the synchronized first time step. In the last two equations, neighbors are calculated using Euclidean distance between the control points of different B-spline curves. Similar to the original interpolation-based method, we can use any interpolation method to calculate the interpolation weights w_i .

5 RESULTS

The dataset that we use to evaluate our B-spline control point interpolation method is generated by a cosmology simulation called `vbbhlight` [13] for solving general relativistic magnetohydrodynamics. The simulation generates particle traces for 2001 time steps. We implemented both our method and the baseline method [3] with Python for a fair comparison. The B-spline approximation is performed using SciPy API to FITPACK, which is a Fortran routine for B-spline fitting and evaluation. We calculate the knot vectors for each curve before fitting by implementing the method described by fast automatic knot placement [16]. Our implementation can be found here¹.

5.1 B-spline Fitting Evaluation

As the first step of our approach, we evaluate the quality of the B-spline approximation on pathlines. The approximation accuracy depends on the knot placement and how we parameterize the curve. We calculated the fitting error as root-mean-squared-error (RMSE) across different integration steps over time. Overall, we achieve RMSE of 1.31×10^{-5} , about 0.000095% of the data range, across all time steps using the 3D B-spline curves with 100 control points parameterized by time.

We compared the fitting accuracy under two different conditions: 4D curves parameterized by the chord length and 3D curves parameterized by time with 100 control points as shown in the left part of Fig. 2, and four different numbers of control points for 3D curves in the right figure. These errors are calculated by interpolating the B-spline curves at the parameters $u_0, u_1, \dots, u_{m-1}$, which corresponds to the time steps in B-spline fitting. Since 4D errors are not comparable to 3D errors, we only calculate the 3D spatial error in the 4D B-spline case. We can observe that 3D curves have fewer errors, especially at the first 750 time steps, when the pathlines have higher curvature in the dataset. The reason for the error difference is that 4D spatial-temporal curves have more complicated geometry because of the additional dimension, which means more control points are needed to achieve a similar spatial error. The comparison between the different number of control points clearly shows that increasing from 10 to 100 control points dramatically decreases the approximation error. However, using more than 100 control points does not decrease the error much, while increasing the computational burden in spline tracing. Since the RMSE of the time step with the largest error is already low enough (0.01, 0.07% of the data range), we choose to use 100 control points for the latter experiments. For other datasets or under different use cases, we may also use a linear regression model as a heuristic method [16] to determine the number of control points.

5.2 Interpolation-based B-spline Tracing Results

In this section, we quantitatively and qualitatively compare the B-spline tracing results with the particle tracing results. For all 83274 pathlines in the dataset, we randomly sample 25% of them as the test data. We start the pathline tracing at different time steps using the sampled test data position. We use the same inverse distance weighting (IDW) interpolation for both methods for a fair comparison. The pathline tracing results are compared with the ground truth test data to calculate spatial RMSE across different time steps.

¹https://github.com/harviu/interp_based_spline_tracing

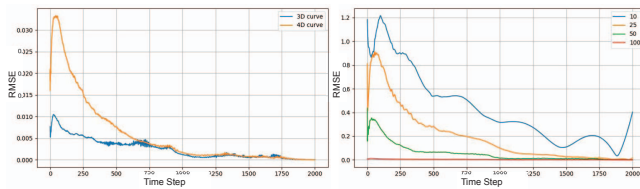


Figure 2: B-spline fitting error in RSME. The left figure compares the 3D curve parameterized by time and the 4D curve parameterized by chord length. The right figure compares the 3D curve using different numbers of control points.

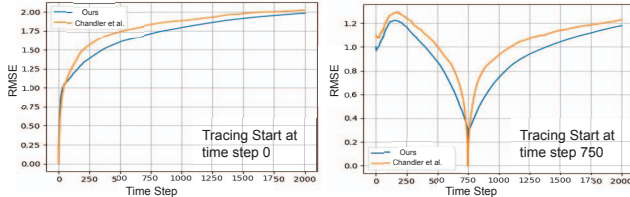


Figure 3: The pathline tracing results that start at different time steps (0 and 750) using our method and the method proposed by Chandler et al. [3]. Our method has a slightly lower error under almost all conditions.

Quantitative results are shown in Fig. 3. Our method and the compared method have a similar error trend when tracing from different starting points. Errors are low around the starting points and increase when the traces advance further. Errors are generally smaller in the first 100 time steps because the data ranges are smaller in these time steps, which is the property of this dataset. Our method has slightly lower tracing error across all time steps, possibly because the B-spline is a smoother representation of the original particle trajectories, and thus removing the fluctuations in the original trajectories could lead to lower error. Next, we show the qualitative pathline tracing results using two different methods in Fig. 4. In Fig. 4 (a), we show three pathlines (first 100 time steps) generated by our method, by the baseline method, and from the original test data. The pathline traced using our method and the baseline method are similar. The errors between the traced pathline and the ground truth pathline are accumulated for later time steps. However, this problem exists for both approaches and can be eased by using a more sophisticated interpolation method. In Fig. 4 (b), we show all the traced pathlines from the test dataset. The color denotes the tracing error at different time steps for each pathline. We can observe similar tracing quality using two different methods. Since it has already been shown [3] that the baseline method has similar tracing accuracy compared to numerical integration methods like adaptive Runge-Kutta 4/5 [2], our method has a comparable tracing error with both the baseline method [3] and numerical integration, and moreover, requires much less computation, which we will show in the next section.

5.3 Timing

We calculate the B-spline fitting time and the interpolation time of 20818 test pathlines. All the experiments are performed on the same machine and with a similar implementation optimization. The particle dataset is assumed to be pre-arranged in a way that particles in the same pathline are close to each other in the memory.

We present the B-spline control points tracing computational time under the conditions of different numbers of control points in Table 1. The number of control points does not influence the B-spline fitting time much. However, the interpolation time increases linearly with the number of control points of interpolation. For the number of control points 100 that we choose for the experiments, the interpolation time is about 10% of the particle interpolation

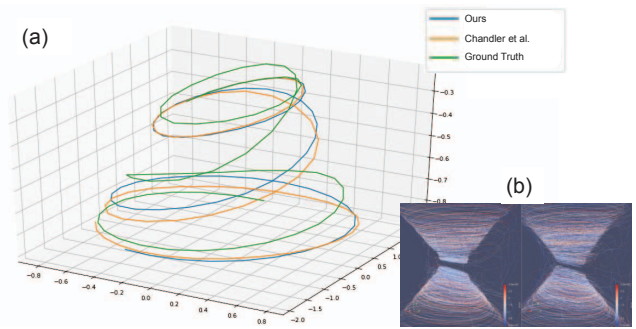


Figure 4: The single pathline tracing results (a) and the tested pathline tracing results (b). Our method is on the left and the baseline approach is on the right. Color denotes the error to the ground truth pathlines. Two methods have similar results both in (a) and (b).

Table 1: B-spline approximation time and control points interpolation time under the conditions of different numbers of control points. The ratio shows the computation time compared to the baseline method.

Number of Control Points	Fitting Time	Interpolation Time	Ratio
10	67.36s	3.19s	0.011
25	67.52s	6.50s	0.023
50	68.80s	11.04s	0.040
100	69.28s	26.71s	0.096
Baseline [3]	-	278.90s	1

time. For most flow datasets, choosing a number of control points similar to this dataset (5% of the number of time steps) can achieve high B-spline fitting accuracy [1]. If the flow data is especially complex, e.g. a lot of fluctuation along the particle trajectories, and we can not reduce the number of control points much, the computation cost of our method could exceed the computation cost of directly interpolating particles. However, this is unlikely for real flow simulations. The B-spline fitting time is much less than that of interpolation and the fitting only needs to be done once for the same dataset. In terms of scalability, the time for the fitting process and tracing for different pathlines increases linearly to the number of pathlines. Since the neighborhood search takes most of the time for interpolating between control points, the time will increase logarithmically to the number of pathlines.

6 CONCLUSION AND FUTURE WORKS

We presented an interpolation-based pathline tracing method for particle simulations on B-spline control points. The B-spline approximation is optimized by using an adaptive knot placement method, which is fast and has controllable errors. The computation time for our method is largely reduced due to the reduction of interpolation iteration, and at the same time, our method achieves similar pathline tracing quality compared to interpolating particles. Besides the usage for pathline tracing, B-splines can also be applied to data reduction or as a proxy representation for neural network training. Our future works will focus on B-spline and neural network representations for the pathlines and their application for feature-driven visualization.

ACKNOWLEDGMENTS

This work is supported in part by US Department of Energy SciDAC program DE-SC0021360, National Science Foundation Division of Information and Intelligent Systems IIS-1955764, and National Science Foundation Office of Advanced Cyberinfrastructure OAC-2112606.

REFERENCES

- [1] R. Bujack and K. I. Joy. Lagrangian representations of flow fields with parameter curves. In *2015 IEEE 5th Symposium on Large Data Analysis and Visualization (LDAV)*, pp. 41–48. IEEE, 2015.
- [2] J. R. Cash and A. H. Karp. A variable order runge-kutta method for initial value problems with rapidly varying right-hand sides. *ACM Transactions on Mathematical Software (TOMS)*, 16(3):201–222, 1990.
- [3] J. Chandler, H. Obermaier, and K. I. Joy. Interpolation-based pathline tracing in particle-based flow visualization. *IEEE Transactions on Visualization and Computer Graphics*, 21(1):68–80, 2015. doi: 10.1109/TVCG.2014.2325043
- [4] C.-M. Chen, A. Biswas, and H.-W. Shen. Uncertainty modeling and error reduction for pathline computation in time-varying flow fields. In *2015 IEEE Pacific Visualization Symposium (PacificVis)*, pp. 215–222. IEEE, 2015.
- [5] G. E. Farin and G. Farin. *Curves and surfaces for CAGD: a practical guide*. Morgan Kaufmann, 2002.
- [6] W. J. Gordon and R. F. Riesenfeld. B-spline curves and surfaces. In *Computer aided geometric design*, pp. 95–126. Elsevier, 1974.
- [7] M. Han, S. Sane, and C. R. Johnson. Exploratory lagrangian-based particle tracing using deep learning. *arXiv preprint arXiv:2110.08338*, 2021.
- [8] F. Hong, C. Bi, H. Guo, K. Ono, and X. Yuan. Compression-based integral curve data reuse framework for flow visualization. *Journal of Visualization*, 20(4):859–874, 2017.
- [9] J. Jakob, M. Gross, and T. Günther. A fluid flow data set for machine learning and its application to neural flow map interpolation. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):1279–1289, 2020.
- [10] J.-Y. Lee and J. Park. Deep regression network-assisted efficient streamline generation method. *IEEE Access*, 9:111704–111717, 2021.
- [11] D. Liu and W. Wang. Topological relation preserving streamline compression based on b-spline curves with bounded error. *Journal of Visualization*, 25(1):111–125, 2022.
- [12] T. McLoughlin, R. S. Laramée, R. Peikert, F. H. Post, and M. Chen. Over two decades of integration-based, geometric flow visualization. In *Computer Graphics Forum*, vol. 29, pp. 1807–1829. Wiley Online Library, 2010.
- [13] J. M. Miller, B. R. Ryan, and J. C. Dolence. vbhlight: Radiation grmhd for neutrino-driven accretion flows. *The Astrophysical Journal Supplement Series*, 241(2):30, 2019.
- [14] F. H. Post, B. Vrolijk, H. Hauser, R. S. Laramée, and H. Doleisch. Feature extraction and visualisation of flow fields. In *Eurographics (State of the Art Reports)*, 2002.
- [15] S. Sahoo and M. Berger. Integration-Aware Vector Field Super Resolution. In M. Agus, C. Garth, and A. Kerren, eds., *EuroVis 2021 - Short Papers*. The Eurographics Association, 2021. doi: 10.2312/evs.20211054
- [16] R. Yeh, Y. S. Nashed, T. Peterka, and X. Tricoche. Fast automatic knot placement method for accurate b-spline curve fitting. *Computer-Aided Design*, 128:102905, 2020.